

KERNEL PANIC

Final Game Design Document

Dylan Overby · Multi-Agent AI for Game Development · Assignment #2 · Due 23 July 2026

Turn-based cyberpunk dueling puzzle · Unity · 2D desktop-UI presentation

Revision notes: this final draft was rebuilt around prototype playtesting of the first draft (designer + friends), then a six-agent adversarial stress-test of the revision. Testers unanimously preferred the Network/Virus "race to the center" dive over the other three dive types combined; the stress-test's top finding was that the restart rules were undefined, which the roguelike run structure below resolves. Changes are flagged inline where they apply, marked like the note below.

REVISED FROM DRAFT 1 Example: sections carrying this marker explain what changed from the Assignment #1 draft and why.

Executive Summary

Kernel Panic is a turn-based cyberpunk roguelike about inheriting your father's computer repair business and the machine in the back room he never let you touch. Your sister runs the front counter and insists the locked machine holds nothing but a nasty virus; you're certain it's more. Every run opens the same way: you dive it, you're not ready, it seals itself shut — and then you get to work, diving customers' infected devices to duel the intrusions inside. Each in-game day opens with diagnostics on every waiting job, so you plan your order before committing resources.

Every duel is a race to connect a signal to the center of a shared grid of rotatable bridge nodes — the same connect-the-pipes fantasy as Watch Dogs' hacking minigame, but turn-based and head-to-head. Each turn you generate RAM, spend it to place and rotate the next node toward the center, and may spend what's left on an ability: arm a trap on the opponent's path, scan your own for theirs, redirect one of their placed nodes, or shield one of yours. Winning is primarily a test of pathing efficiency — a raw execution skill independent of loadout; abilities disrupt the opponent's route or protect yours, but never replace reading the grid well.

Win condition: reach the fixed finale day strong enough to win the duel against your father's machine and finally see what's inside — the AI companion he built for his son, sealed away until the son could stand on his own.

Loss condition: one rule, everywhere: Neural Strain reaching zero — whether zeroed instantly by a lost duel or bled dry by messy wins — ends the run. Unlocked abilities persist across runs; currency, RAM, and Neural Capacity reset. Each failed run advances the story.

Game Mechanics

Core Loop

REVISED FROM DRAFT 1 Draft 1 had four separate dive mechanics; playtesting showed the race-to-center duel was where all the fun lived, so the game was rebuilt around it. The agent stress-test then flagged Draft 2's arc-restart rules as undefined with every reading broken — the roguelike run replaces them with a single rule.

A run spans ten in-game days. Each day opens with diagnostics on its three jobs; the player chooses the order, knowing Neural Strain is shared across everything. Each job runs: Analyze (a diagnostic readout flags the opponent's dominant behavior, e.g. "Virus launches Redirect often"), Adjust Build (equip unlocked abilities into the Neural Capacity-limited loadout, informed by Analyze), then Dive (the duel). Clearing a day grants one upgrade for the rest of the run: raise Neural Capacity or RAM. Days scale harder — larger grids, more aggressive opponents — the escalating-stakes shape Balatro uses across its antes. Neural Strain is the run's single life bar: messy wins chip it, an outright lost duel zeroes it instantly, and zero — by any means — ends the run.

The Duel

REVISED FROM DRAFT 1 Draft 1's duel was real-time. Ability trades were becoming about reflexes rather than decisions, so the final design is turn-based — same race, same stakes, room to plan.

The grid is built from rotatable bridge nodes: each is a connector piece rotated to link toward the center, and both sides race to complete a connected signal path from their entry point to the middle first. Each turn the player generates RAM, spends it to place and rotate their next node, and may spend the remainder on one ability against either grid. A player who routes well can win on pathing alone; abilities support the contest without replacing it. The opponent runs on simple scripted logic — not a live agent — reacting to the player's pace and ability use with a mirrored kit. Reaching the center first wins; a loss costs Neural Strain.

Resources

REVISED FROM DRAFT 1 Draft 1 tracked four skill trees and four upgrade benches. These collapsed into three shared resources — a major simplification of both content volume and UI surface.

Resource	Role	Gained	Spent
Neural Strain	The run's life bar	Restored at the start of each run	Chipped by messy wins; zeroed instantly by a lost duel. Zero, by any means, ends the run.
RAM	Turn-by-turn action currency	Generated automatically every turn; stat raised via end-of-day choice, resets between runs	Buys node placement/rotation and ability activations — movement and abilities compete for the same pool
Neural Capacity	Run-scoped loadout size	Raised via the end-of-day upgrade choice; resets between runs	Caps how many unlocked abilities can be equipped per dive

Abilities & Progression

Every successful dive unlocks one new ability drawn at random from the full pool, regardless of tier — tier governs RAM cost to activate, not eligibility to unlock, so a player can win a Tier 3 ability long before they can afford to fire it: a bet on future growth, not a locked door. Each unlock grants one copy; using an ability in a duel consumes a copy, and further copies of anything already unlocked can be purchased before a dive.

Run carryover: unlocked abilities are the only thing that persists between runs — currency, copies, RAM, and Neural Capacity all reset. Options accumulate; power does not. A veteran starts run five with a wide toolbox and the same tiny loadout as run one, so experience is expressed through smarter equip choices, not stat inflation.

Tier	Ability	Effect	RAM cost to activate
1	Arm Node	Trap a node on the opponent's path; routing through it costs them a turn.	Low
1	Scan	Reveal any armed nodes on your own path. Direct counter to Arm Node.	Low
1	Redirect	Change the connection on an opponent's placed node, undoing progress.	Low
1	Shield	Lock one of your own nodes against Arm Node and Redirect for a turn.	Low
2	Overload	Disable one opponent ability for their next turn.	Medium
2	Overclock	Boost your RAM generation on your next turn — a tempo play.	Medium
3	Firewall	Temporary immunity to Arm Node and Redirect for one turn.	High
3	Backdoor	Instantly clear a trap on your own node without needing Scan.	High

The eight base verbs. Each ships with 2–3 parameter variants (Arm Node II traps two nodes; Deep Scan also reveals the opponent's next ability; Long Shield holds two turns), widening the unlock pool to roughly 24 items with no new engine work — variants are config rows, not new mechanics.

Structure, Presentation & Finale

REVISED FROM DRAFT 1 Draft 1 had a persistent 2.5D shop with purchasable machines, employees, and decoration — all cut; the game is a 2D desktop-themed UI, the largest scope reduction in this revision. The stress-test flagged the finale's one-off game-over as incoherent with the soft-failure loop and the father reveal as having no build-up; the roguelike structure resolves both.

Every run opens with a scripted, unwinnable dive into the father's machine — this is the tutorial. The player learns the controls in a duel they cannot win, the machine seals itself, Neural Strain zeroes, and day one begins. The loop's shape is demonstrated diegetically before the first real job, and no bespoke tutorial system exists: it is one scripted tier-0 duel config.

The finale falls on Day 10, every run, whether the player is ready or not — a fixed date makes it possible to judge in playtesting exactly how strong a player should be by then, and gives every run a deadline. Mechanically it is the same duel engine at the top of the difficulty curve. Losing it is not a special case: strain zeroes, the run ends, same rule as everywhere else. Winning finally opens the machine — the AI companion the father built for his son, sealed until the son could stand on his own.

Failed runs advance the story. The sister's theory — it's just a nasty virus — erodes run by run as the machine behaves less and less like malware, and fragments of the father surface with each attempt: tagged dialogue keyed to run count, delivered through the existing Outcome & Dialogue pipeline and a small set of reusable illustrated stills. Losing is how the player learns who their father was.

AI Architecture

Every agent below runs during development, not during play. Each produces content in a defined format, baked into a static library the shipped game draws from and lightly randomizes at runtime. Kernel Panic makes no API calls while it is being played.

Customer/Encounter Generator — the “One Wow.” Produces one structured JSON profile per job: customer name, device, flavor text, opponent difficulty tier, and dominant-ability tag. Feeds both the diagnostic readout and the duel's starting conditions — the player meets a specific character with a specific, telegraphed opponent on every job.

Arc Composer. Produces structured config values (grid size, opponent RAM, ability-use frequency) for each day of the fixed escalation curve, ending in the Day 10 finale config.

Validation. Produces a pass/fail report per arc-tier configuration, stress-testing sample duels so the opponent's scripted logic cannot force an unwinnable state.

Outcome & Dialogue. Produces short tagged text strings (win, loss, day-start, run-start, run-end — the latter keyed to run count for the sister's evolving virus theory and the father fragments) covering duel outcomes and all front-counter dialogue.

Art Lead. Produces 2D assets — desktop UI chrome, ability icons, portraits, cutscene stills — from purchased icon/UI packs, with supplementary pieces generated in Higgsfield where packs fall short.

Lore-master. Produces an approve/revise note per item, reviewing generator and art output against a maintained setting bible before anything ships.

Pipeline order: the four content agents produce in parallel; Validation checks arc-tier fairness; Lore-master reviews the rest before anything enters the shipped library.

Technical Strategy

REVISED FROM DRAFT 1 Cutting the 2.5D shop removed the entire 3D pipeline (Meshy) from the project, and cutting three dive types shrank the content plan — the total budget below is roughly a third smaller than Draft 1's.

Kernel Panic is built in Unity as a 2D UI-driven project. All visual content is 2D: purchased UI/icon packs for the bulk, Higgsfield for the pieces packs cannot cover. All generation happens in pre-production. Figures below count input and output tokens together, since both draw against plan usage.

Agent / task	Model	Calls	Tokens per call (in + out)	Estimated total
Customer/Encounter Generator	Sonnet	60	$\sim 1,200 + \sim 1,500 = 2,700$	162,000
Arc Composer	Sonnet	12	$\sim 800 + \sim 500 = 1,300$	15,600
Outcome + Dialogue	Sonnet	30	$\sim 600 + \sim 200 = 800$	24,000
Lore-master	Sonnet	80	$\sim 900 + \sim 250 = 1,150$	92,000
Validation	Haiku	240	$\sim 300 + \sim 400 = 700$	168,000
Development (build)	Opus	~ 50 turns	$\sim 8,000$ average per turn	$\sim 400,000$
Total across all models				$\approx 861,600$ tokens

The development row is the least predictable line — coding sessions resend conversation history every turn — and is controlled by clearing context between subtasks rather than by call count.

Anthropic publishes no specific token ceiling for Max plan usage — only that Max 20x provides twenty times Pro's per-session usage, alongside weekly caps including a separate, tighter cap for Sonnet. This budget is therefore sized in absolute tokens and spread across a multi-week pre-production rather than planned against a published number.

Constraints & Scope

- **One person, remaining weeks.** The single-mechanic, UI-only redesign exists specifically to fit that window. Exact balance numbers (Neural Strain costs, RAM generation rate) are deliberately not finalized — they require a working build to tune honestly, and this document states the levers rather than faking precision.
- **Image credits are capped.** Higgsfield allows a fixed number of generations monthly; Art Lead builds from purchased packs for this reason and generates only supplementary pieces.
- **Context windows are fixed at 200K tokens.** Lore-master reviews against a condensed setting-bible summary, not the full history of prior generations, which would grow every prompt linearly with the library.
- **Validation loops must terminate.** The generate-and-check cycle caps at three retries before a failing arc-tier is flagged for manual review.
- **Sonnet has its own weekly sub-cap.** Validation runs on Haiku because it is the highest-call-count agent; routing that volume away from Sonnet keeps the heaviest load off the most constrained budget.